Sprawozdanie z Projektu 1

Analiza Dźwięku w Dziedzinie Czasu

Przedmiot: Analiza i Przetwarzanie Dźwięku (AiPD)
Autor: Kacper Tomczyk, Nr indeksu: 333166
Data: 2 Kwietnia 2026
Repozytorium: `https://github.com/kacpert20/AUDIO`

Cel projektu i zastosowane narzędzia

Celem projektu było stworzenie od podstaw aplikacji okienkowej zdolnej do ekstrakcji i analizy cech sygnału audio w dziedzinie czasu, a także zaimplementowanie heurystycznego klasyfikatora rozróżniającego mowę od instrumentów muzycznych.

Cała matematyka została zaimplementowana ręcznie przy użyciu wektorowych operacji z biblioteki *numpy*. Interfejs graficzny oparto na frameworku *PyQt6* oraz bibliotece *pyqtgraph*, co zapewniło wysoką wydajność renderowania wykresów. Pliki dźwiękowe ładowane są za pomocą *scipy*.

Architektura Aplikacji i Interfejs

Program składa się z trzech odseparowanych modułów:

- **audio_io.py**: Obsługa wejścia/wyjścia (wczytywanie plików .wav).
- **math_engine.py**: Silnik analityczny wykonujący podział na ramki, obliczanie energii, głośności, ZCR, estymację tonu podstawowego (Autokorelacja i AMDF) oraz obliczający statystyki klipu. Znajduje się tu również logika decyzyjna klasyfikatora.
- **gui_main.py**: Zarządzanie interfejsem użytkownika, synchronizacja osi wykresów, malowanie regionów dźwięcznych i eksport wyników do CSV.

Rysunek 1: Interfejs aplikacji analizującej sygnał audio.

Instrukcja Obsługi i Zbiór Danych

Instrukcja uruchomienia programu

Aplikacja została przystosowana do działania w środowiskach Windows oraz systemach typu UNIX. Aby poprawnie uruchomić program, należy upewnić się, że w systemie zainstalowany jest interpreter języka Python w wersji 3.x. Proces uruchomienia sprowadza się do pobrania zawartości repozytorium oraz instalacji wymaganych zależności zdefiniowanych w kodzie:

1. W konsoli należy wykonać instalację pakietów poleceniem: `pip install numpy scipy pyqtgraph PyQt6 pandas`.
2. Uruchomić główny skrypt poleceniem: `python gui_main.py`.

Po uruchomieniu użytkownik korzysta z interfejsu graficznego, ładując pliki z rozszerzeniem `.wav` za pomocą przycisku w lewym górnym rogu.

Baza próbek testowych i eksport do CSV

W strukturze katalogów projektu zaimplementowano własną bazę plików dźwiękowych (`/znormalizowane/16` oraz `/znormalizowane/instrumenty`). Po załadowaniu próbki system generuje podsumowanie na prawym panelu statystyk. Przycisk **Eksportuj wyniki do CSV** pozwala na błyskawiczne zapisanie wyliczonych metryk do arkusza. Umożliwiło to zautomatyzowane zbudowanie bazy danych, która posłużyła do optymalizacji klasyfikatora.

Zestawienie Wyników Eksperymentów

Przeprowadzono testy na zbiorze znormalizowanych i nieznormalizowanych plików .wav, zawierających nagrania ludzkiej mowy oraz dźwięków instrumentów. Poniżej znajduje się

podsumowanie poprawnych wyników klasyfikacji.

Przypadki poprawnego działania algorytmów

- **Ciągłe samogłoski i dźwięki muzyczne:** Algorytmy wyznaczania F_0 (Autokorelacja i AMDF) radzą sobie z czystymi, powtarzalnymi falami. Dla samogłosek oraz uderzeń struny gitary wykres F_0 tworzy płaską linię.
- **Wykrywanie alikwotów:** Wykresy precyzyjnie potwierdziły zjawiska fizyczne, np. szarpnięcie struny E2 nad 12. progiem zaowocowało stabilnym odczytem F_0 w okolicach 165 Hz (drugi alikwot).
- **Klasyfikacja:** Opracowana na bazie zebranych statystyk nieliniowa funkcja decyzyjna skutecznie izoluje mowę od instrumentów.

Rysunek 2: Poprawna estymacja F_0 dla szarpniętej struny gitary.

Wyzwania implementacyjne i ograniczenia

Niektóre funkcjonalności, takie jak detekcja ciszy czy idealne wyodrębnienie tonu podstawowego F_0, nie działają perfekcyjnie dla każdej próbki. Dostrojenie parametrów stanowi zawsze kompromis pomiędzy czułością a odpornością na zakłócenia. Pełne zniwelowanie tych problemów wymagałoby zastosowania zaawansowanych filtrów adaptacyjnych lub przejścia w dziedzinę częstotliwości, co wykracza poza zakres bieżącego etapu prac.

Elementy rozszerzające

Aby wyczerpać wyższe kryteria oceny, wprowadzono autorskie modyfikacje:

1. **Nieliniowy Klasyfikator:** Sztywne progi decyzyjne dla parametru LSTER, proponowane często w literaturze, okazywały się nieskuteczne przy testach na własnym zbiorze nagrań. W celu rozwiązania tego problemu, wykorzystano wbudowany w

aplikację moduł eksportu do CSV, tworząc pełną bazę statystyk dla wszystkich zebranych plików. Analiza rozkładu danych na wykresie punktowym ujawniła specyficzną, krzywolinijną korelację. W toku empirycznego testowania dopasowano do zbioru danych parabolę decyzyjną:

$$LSTER < -0.1(Entropy - 3)^2 + 0.8 \tag{1}$$

Dźwięki znajdujące się poniżej wyznaczonej krzywej klasyfikowane są jako mowa, a te powyżej jako instrumenty. Rozwiązanie to drastycznie zwiększyło skuteczność rozpoznawania sygnału.

2. **Autorski parametr "Czas Ataku":** Zaimplementowano cechę wyliczającą czas do osiągnięcia maksymalnej amplitudy sygnału, wspierającą analizę dynamiki.

3. **Wizualizacja Widma (Spektrogram):** Dodano wykres obrazujący dystrybucję energii w dziedzinie częstotliwości.

Podsumowanie

Analiza w dziedzinie czasu dostarcza skutecznych, acz wrażliwych na szum i amplitudę metod ekstrakcji cech. Zbudowana aplikacja pozwala na weryfikację zjawisk akustycznych, a testy obnażyły słabości prostych estymatorów przy sygnałach o dużym zaszumieniu i niskiej energii. Projekt spełnia wszystkie założenia i stanowi stabilny fundament analityczny.

Rysunek 3: Przykładowe działanie - dźwięk fletu.